Cyber Threats and their Mitigations

Technical Communication Presentation

Group 4:

Adesh Palkar	23114004
Nisarg Prajapati	23114073
Utkarsh Kumar	23114101
Tanay Kapadia	23323044
Parth Baranwal	23114076

April 16, 2026

Source: Pinterest stock image

ppt@techcomm:~$ cat Table_of_Contents.txt

1. Overview of Topic
2. Case Study
3. Problem Statement
4. Related Work
5. Existing Solution
6. Proposed Solution
7. Conclusion
8. References

Source: Pinterest stock image

The Modern Cyber Threat Landscape

- Global cybercrime cost has reached **Trillions** of USD.
- Threat actors range from opportunistic actors to **nation-state APTs** and organised crime syndicates
- Attack surface spans: cloud, CI/CD pipelines, open-source ecosystems, IoT
- Modern attacks are *targeted*, *persistent*, and **systemic**

Framing Note

Adversaries **chain techniques** across the kill chain. We survey six major threat classes before narrowing to our focal threat: **software supply chain attacks**.

Threat 1 : Malware & Ransomware

Attack Vector

Malicious software designed to **encrypt, steal, or destroy** data. Attackers demand payment to restore access.

Key Mitigations

- Antivirus & Endpoint Security
- Secure, offline backups

Threat 2 : Phishing & Social Engineering

Attack Vector

Deceptive messages used to trick people into revealing passwords or installing malware. It is the starting point for most cyberattacks. More than 90% of cyberattacks are phishing attacks.

Key Mitigations

- Multi-Factor Authentication
- Email filtering & security

Threat 3 : Denial of Service (DDoS)

Attack Vector

Flooding a service with traffic until it crashes or becomes unresponsive, preventing **real users** from accessing it.

Key Mitigations

- DDoS Protection limits & filters
- Redundant infrastructure

Threat 4 : Insider Threats

Attack Vector

Attacks coming from **within** an organization. Disgruntled or careless employees misuse their authorized access to steal or destroy data.

Key Mitigations

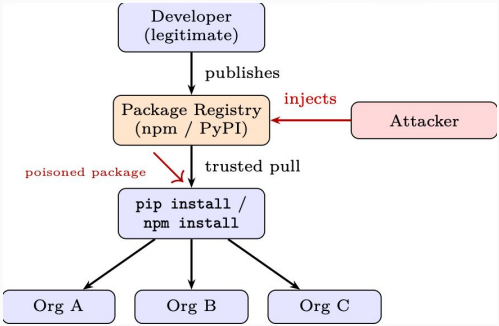
- Restrict access (Least Privilege)
- Monitor user activity

Threat 5 — Software Supply Chain Attacks

What Makes This Categorically Different?

Instead of targeting **your system directly**, adversaries compromise a **trusted third party** — like an open-source library or registry. The threat propagates *silently* to downstream consumers. **The attack vector is trust itself,** bypassing traditional defenses.

Supply Chain Attack Path



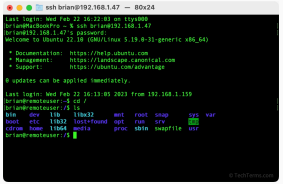
XZ Utils Backdoor (2024)

What is XZ Utils?

- Widely used open-source compression utility in Linux.
- Provides the `liblzma` library used by many systems.

What happened?

- Malicious code was inserted into ver 5.6.0 and 5.6.1 [2].
- Hidden through build-time logic and release files, making it difficult to notice.
- Designed to affect systems linked with sshd / OpenSSH, creating a possible unauthorized access path into critical Linux server environments.



Source:

techterms.com/definition/ssh

Why is it important?

- A trusted low-level dependency became a possible attack path for critical systems.
- Because XZ Utils is used by many environments, compromise could have spread risk widely across Linux systems and affected many downstream software components.

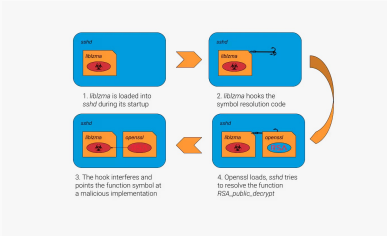
Attack Path and Security Insight

How did the attack work?

- The attacker targeted a trusted upstream package in the software ecosystem.
- Malicious behavior was inserted in the upstream release/build process itself at source.
- Compromise could spread through normal dependency usage across downstream systems.

Security insight

- This is a software supply chain attack [3].
- It exploited implicit trust in the package ecosystem and software supply chain.
- One compromised dependency can affect many downstream users across environments.



Source: akamai.com/blog/security-research

Key message

In supply chain attacks, trust becomes the attack surface.

Problem Statement

“The widespread blind trust in unverified open-source dependencies exposes organizations to devastating software supply chain attacks.”

Technical meaning

- Modern software relies heavily on third-party packages and components.
- These dependencies are often trusted based on popularity or reputation.
- Many organizations do not fully verify code changes, release artifacts, maintainer activity, and transitive dependencies before adoption or deployment.

Result

- One compromised package can expose many systems at once across organizations.
- This creates large-scale risk across interconnected software environments.

Why This Matters

Broader impact

- Modern applications depend on complex third-party dependency chains.
- Risk spreads from upstream packages to many downstream systems.
- Attacks may remain hidden inside trusted updates or build logic paths.

What is needed?

- Stronger dependency verification across critical software components and releases.
- Robust artifact integrity checks during releases and deployment workflows.
- Better maintainer risk awareness and review across project ownership and trust.
- Continuous supply chain security monitoring practices across software ecosystems.

Final takeaway

Trust in software dependencies must be continuously verified, not automatically assumed across modern software supply chains.

Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages

— Ruian Duan et al., NDSS 2021 [1]

What problem does the paper address?

- Modern software heavily depends on third-party packages from registries like PyPI.
- These ecosystems assume implicit trust, which attackers exploit to inject malware.

What does the paper aim to do?

- Provide systematic comparison of security weaknesses in package ecosystems.
- Identify broken trust assumptions and structural vulnerabilities enabling attacks.
- Build an automated pipeline (MALOSS) to detect malicious packages at scale.

Move from reactive, case-by-case fixes to a scalable, data-driven analysis of supply chain threats.

Source: Duan et al.[1], NDSS 2021 (Introduction, Methodology)

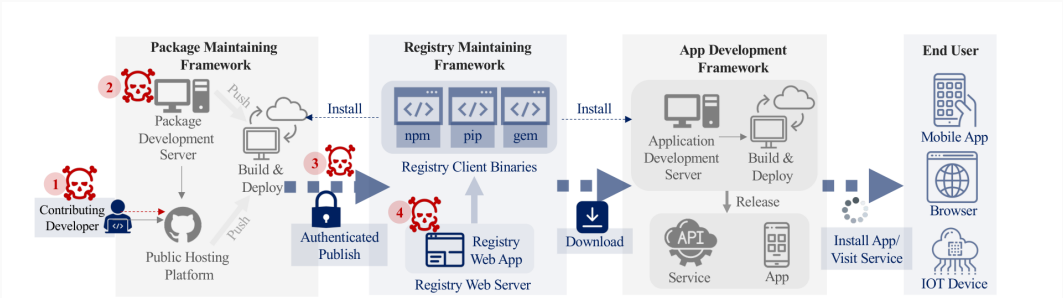


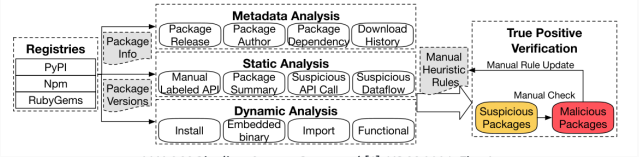
Fig. 1: Simplified relationships of stakeholders and threats in the package manager ecosystem.

Source: Duan et al.[1], NDSS 2021, Fig. 1.

Flow & Security Insight

- Packages flow: developers → registries → applications → users.
- Attacks exploit upstream components such as packages, where trust is assumed.
- Impact spreads downstream across the ecosystem, affecting applications and users.

MALware detection for **O**pen **S**ource **S**oftware



MALOSS Pipeline, Source: Duan et al.[1], NDSS 2021, Fig. 4.

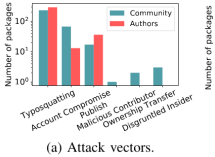
1. **Metadata analysis:**
   - Flags suspicious packages using authors, releases, dependencies, and embedded bins.
   - Helps detect typosquatting, author mismatch, and links to known malware.
2. **Static analysis:**
   - Scans source code for risky APIs: network, filesystem, process, and code generation.
   - Detects suspicious flows such as *secret file* $\rightarrow$ *network exfiltration*.
3. **Dynamic analysis:**
   - Runs packages in Docker sandboxes during install, import, and function execution.
   - Traces DNS/IP activity, file access, and process creation using Sysdig.
4. **Human-in-the-loop verification:**
   - Manual triage verifies flagged packages, confirms malware, and refines heuristic rules.
   - Reduces false positives and improves the accuracy of the pipeline iteratively.

Source: Duan et al.[1], NDSS 2021, Sec. III-B and Table III.

Key findings from the paper

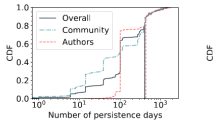
- MALOSS analyzed **over 1 million packages** across PyPI, Npm, and RubyGems.
- It identified **339 new malicious packages**; **278** were confirmed and removed by maintainers.
- **Three** removed packages had **more than 100K downloads** and received CVEs.
- **Typosquatting** and **account compromise** were the most common attack vectors.
- About **20%** of malware stayed online for **over 400 days**, showing slow detection and removal.

Source: Duan et al.[1], NDSS 2021, Abstract and Sec. IV.



(a) Attack vectors. (b) Malicious behaviors.

Fig. 8: Breakdown of malware by attacks and behaviors.



(a) CDF of persistence days. (b) CDF of downloads.

Fig. 9: The distribution of number of persistence days and number of downloads for malware.

Source: Duan et al.[1], NDSS 2021, Fig. 8,9.

Actionable mitigations

- **Registry maintainers:** enforce MFA, automated vetting, typo detection, and fast advisories.
- **Package maintainers:** secure infrastructure, restrict access, and use safe workflows.
- **Developers:** pin dependencies, review install scripts, and treat updates as security events.
- **Ecosystem response:** notify dependents quickly and control releases of popular packages.

Limitations

- Static/dynamic analysis may produce false positives and miss hidden behaviors.
- Dynamic analysis depends on execution paths, trigger-based malware can evade detection.
- Large dependency graphs make complete analysis difficult.
- Manual verification is required, limits full automation.
- Attackers can evade detection using obfuscation or delayed execution.

Source: Duan et al.[1], NDSS 2021, Sec. III-B, Sec. IV and Sec. V.

Almost every sophisticated modern codebase makes use of **dependency managers**. Node has `npm`, Python's `pip` uses the Python Package Index; Composer and RubyGems similarly exist for PHP and Rust, respectively.

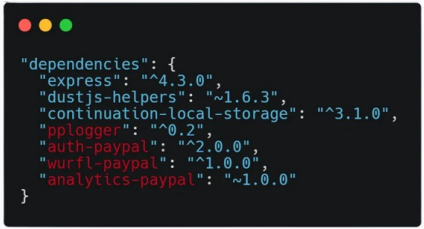
A dependency manager cannot possibly guarantee malware-free code. Over 3.1 million packages are available in the main `npm` registry!

Most dependency managers also allow publishing **private packages**, which means organizations can rely on them for all of their dependencies.

Dependency managers have different tie-breaking rules when a regular `install` command finds a public as well as a private repository. For example, `pip` defaults to a package with a higher version number.

At the time of creation, it can be ensured that there are no clashing public packages. However, they can always be claimed at any instance by a benign or malicious programmer.

Another way in which a similar issue can arise is through abandoned names.



Source: medium.com/@alex.birsan/4a5d60fec610

Alex Birsan, an ethical hacker, found this in a `package.json` file on a PayPal repository hosted publicly on GitHub. [3] He found more such `json` files hidden in JavaScript build files.

Security Analysis of Dependency Confusion for Third Party Components: A Risk Assessment Framework Based on Source Code Tree Matching [2]

- Manual detection for dependency confusion is definitely not feasible, especially when obfuscation is involved as well.
- Currently, a combined analysis of the SBOM and Config files is done as an industry standard. Extracting the desired product and version name from both of these sources and comparing them can lead to capturing obvious discrepancies.
- The paper presents source code analysis as a tool to detect and assess vulnerabilities, which can work without the above documents and can also handle injections and obfuscations.

Method

1. **Obtain source**: Traverse the root directory to analyze the code structure. Use source codes whenever possible, although decompilation is also supported.
2. **Generate AST**: The source code is parsed into an abstract syntax tree. The paper focuses largely on Python, for which this task can be done via the `ast` module.
3. **Optimize the tree**: 123 lines of a sample source code generates a tree with 5377 nodes. A lot of nodes not helpful are removed or replaced by alternatives.
4. **Comparing ASTs**: The generated AST is compared with an AST of the different components of the source code. Breadth-first technique is used for the same.

The paper demonstrates that we are able to realize an accurate (96.00%) mapping between Python code files (or bytecode) with PyPI packages. However, the testing is done in a very limited and synthesized environment.

eBPF — Extended Berkeley Packet Filter

eBPF allows us to run sandboxed programs directly inside the Linux kernel.

Runtime Profiling (Baselines)

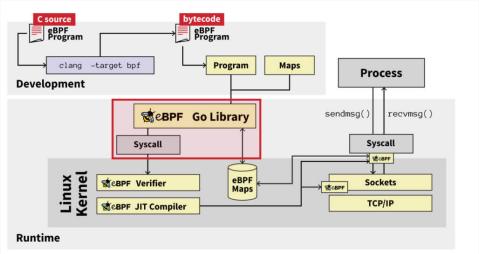
It monitors system behaviour at the source, in real-time.

Deep Packet Inspection (Visibility)

It inspects network packet payloads *before* encryption and detects if a trusted dependency is exfiltrating sensitive data under the hood.

Execution Guarding

Via `LSM` (Linux Security Modules) hooks, eBPF can block unauthorised `execve()` calls.



Source: redcanary.com/blog/threat-detection/ebpf-malware

Immutable Logging

Running in kernel-space, eBPF is invisible to user-space malware. Even with root access, an attacker cannot silence the eBPF reporter.

Limitation: Input Dependency in Dynamic Analysis

Dynamic analysis based tools suffer with **Input Dependency** — if specific input conditions are not met, malicious execution paths can go **undetected**.

Static Analysis

We can use static analysis to compare binary subtasks against behavioral patterns of **known malware families**, identifying potential threats **without executing the code**.

- No runtime environment needed
- Can catch threats before any execution occurs

Symbolic Execution

To solve the input dependency problem, we use **symbolic execution** to mathematically evaluate execution results for **every possible input case**.

- Explore all feasible code paths exhaustively
- Detect trigger based or condition gated malware

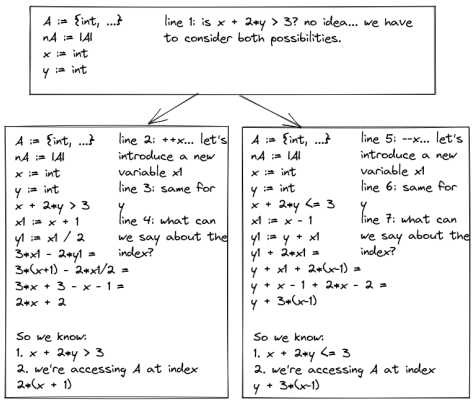
Using symbolic execution, we will end up with a SMT equation for each exit point of the code.

Database Comparision

By comparing the equation with the corpus of equations of known malicious software, we can detect already known forms of malware.

Limitation

Naively comparing the equation with the corpus could be computationally heavy. To avoid this problem, better heuristic algorithms for matching SMT equaions need to be devised.



Source: unwoundstack.com/blog/concrete-example-of-symex.html

Conclusion

- Modern cyber threats are increasingly sophisticated, interconnected, and persistent.
- Software supply chain attacks exploit **implicit trust** in dependencies rather than system vulnerabilities.
- Case studies like **XZ Utils** demonstrate how a single compromised component can impact **entire ecosystems**.
- Existing solutions provide visibility, but no single approach is sufficient to fully mitigate these threats.

- R. Duan, O. Alrawi, R. P. Kasturi, R. Elder, B. Saltaformaggio, and W. Lee, ”Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages,” in *Proceedings of the Network and Distributed System Security (NDSS) Symposium*, 2021. [Online]. Available: https://dx.doi.org/10.14722/ndss.2021.23055
- National Vulnerability Database (NVD), ”CVE-2024-3094 Detail,” National Institute of Standards and Technology (NIST), 2024. [Online]. Available: https://nvd.nist.gov/vuln/detail/CVE-2024-3094
- Red Hat, ”Understanding Red Hat’s response to the XZ security incident,” Apr. 30, 2024. [Online]. Available: https://www.redhat.com/en/blog/understanding-red-hats-response-xz-backdoor

- Cybersecurity and Infrastructure Security Agency (CISA), ”Shields Up: Guidance for Families.” [Online]. Available: https://www.cisa.gov/shields-guidance-families
- Y. Ding, W. Tian, and B. Cui, ”Security Analysis of Dependency Confusion for Third Party Components: A Risk Assessment Framework Based on Source Code Tree Matching,” 2024. Available: https://doi.org/10.21203/rs.3.rs-5261746/v1
- A. Birsan, ”Dependency Confusion: How I Hacked Into Apple, Microsoft, and Dozens of Other Companies,” 2021. [Online]. Available: https://medium.com/@alex.birsan/dependency-confusion-how-i-hacked-into-apple-microsoft-and-dozens-of-other-companies-4a5d60fec610

Name	Contribution
Adesh Palkar	Concise overview of modern cybersecurity threats and their practical mitigations
Nisarg Prajapati	XZ Utils case study, attack-path analysis, problem statement, broader impact discussion.
Utkarsh Kumar	Research Paper 1 - MALOSS Pipeline, Key Findings, Mitigation Strategies & Limitations
Tanay Kapadia	Dependency Confusion - Overview, Origin, Solution and its Limitation
Parth Baranwal	Exploring existing solutions and future solution possibilities.

Session Terminated.

Q&A / Discussion